

面向实希尔伯特空间的内积-范数化简 Tactic

从 KM 迭代中的不等式证明出发

虞健 唐煦江 张佳函

浙江大学 AI4MATH

2026 年 7 月



汇报路线

- ① 问题与动机
- ② Tactic 设计
- ③ 论文应用与实现

围绕一个论文中的关键下降不等式，展示问题、设计、验证与实现。

① 问题与动机

② Tactic 设计

③ 论文应用与实现

问题来自 KM 迭代的下降不等式

在 Krasnosel'skii–Mann 迭代的收敛证明中，需要验证

平方距离的下降估计（论文中的关键不等式）

$$\|(1-a)x + aTx - y\|^2 \leq \|x - y\|^2 - a(1-a)\|Tx - x\|^2.$$

其中 T 非扩张、 $Ty = y$ ，且 $a \geq 0$ 。

- 数学上，这是凸组合的平方范数恒等式加非扩张性；
- 在 Lean 中，需要把范数平方改写为内积、双线性展开，再完成实数多项式化简；
- 这种模式在希尔伯特空间的优化算法形式化中反复出现。

先看数学结构：一条恒等式承载了主要展开工作

令 $u = x - y$ 、 $v = Tx - y$ ，则 KM 步可改写为 $(1 - a)u + av$ 。核心恒等式为

凸组合的平方范数恒等式

$$\|(1 - a)u + av\|^2 = (1 - a)\|u\|^2 + a\|v\|^2 - a(1 - a)\|v - u\|^2.$$

- 非扩张性给出 $\|v\| \leq \|u\|$ ，从而控制中间项；
- $v - u = Tx - x$ ，恰好得到所需的残差项；
- 纸面上是一行恒等式加一个不等式，在 Lean 中却涉及内积展开与实数代数规范化。

原始 calc 证明：展开与重排成为主要负担

```
have h_expand :  
  ||(1-a) • (x-y) + a • (T x-y)|| ^ 2 =  
    (1-a) * ||x-y|| ^ 2 + a * ||T x-y|| ^ 2  
  - a * (1-a) * ||(T x-y)-(x-y)|| ^ 2 := by  
rw [← real_inner_self_eq_norm_sq ...]  
simp only [inner_add_left, inner_add_right,  
  inner_sub_left, inner_sub_right,  
  real_inner_smul_right, real_inner_comm]  
ring
```

- 要显式列出范数-内积改写；
- 要手动展开内积的加法、减法、数乘规则；
- 最后还要组织加权不等式与多段 calc。

证明的数学核心被样板化代数步骤淹没。

- ① 问题与动机
- ② Tactic 设计
- ③ 论文应用与实现

设计目标：为实希尔伯特空间封装两类重复推理

适用空间

```
variable {H : Type*} [NormedAddCommGroup H] [InnerProductSpace ℝ H]
```

inner_norm

处理等式：

- 平方范数 $\leftrightarrow$ 自内积；
- 内积的双线性展开；
- 实系数多项式标准化。

inner_bound

处理不等式：

- 范数非负、三角不等式；
- Cauchy–Schwarz 型内积界；
- 利用给定的已知界（辅助不等式）
做平方、线性/非线性推理。

限制为实空间：复内积的共轭线性使最终表达式不再是同一类交换多项式。

inner_norm: 从几何表达式到多项式恒等式

处理管线

$\|u\|^2 \implies \langle u, u \rangle \implies \text{按双线性展开} \implies \text{实数多项式规范化.}$

- 范数平方改写: 使用 `real_inner_self_eq_norm_sq`;
- 内积展开: 处理加法、减法、相反数与实数数乘;
- 交叉项统一: 使用实内积交换律;
- 代数闭合: 交给 `ring` 验证最终恒等式。

这一设计只针对实希尔伯特空间; 复空间中的共轭线性不能直接归约为同一类交换多项式。

inner_norm: 核心实现代码

```
open Lean Elab Tactic

namespace LeanHilbert

elab "inner_norm" : tactic => do
  evalTactic (← `(tactic|
    simp only [← real_inner_self_eq_norm_sq,
      inner_add_left, inner_add_right, inner_sub_left, inner_sub_right,
      inner_neg_left, inner_neg_right, real_inner_smul_left,
      real_inner_smul_right, real_inner_comm, inner_zero_left,
      inner_zero_right, smul_eq_mul, neg_mul, mul_neg] <;>
    ring))

end LeanHilbert
```

simp only 负责语义保持的重写，ring 负责纯代数归一化。

inner_bound: 先匹配标准界, 再组合已知界

两层设计

- ① 规则层: 直接匹配范数非负、三角不等式、Cauchy–Schwarz 上下界;
 - ② 组合层: 用户以 inner_bound $[h_1, \dots, h_n]$ 给出已知界, tactic 自动做平方比较或交给 nlinarith 联合推理。
- 一个已知界: 优先尝试 $\|x\| \leq \|y\| \Rightarrow \|x\|^2 \leq \|y\|^2$;
 - 多个已知界: 将它们作为实数不等式系统的条件;
 - 因此 tactic 保持可控: 需要额外信息时由用户显式提供。

inner_bound: 核心实现代码

```
syntax "inner_bound" ("[" term,* "]" )? : tactic

elab_rules : tactic
| \ (tactic| inner_bound) => do
  evalTactic (← \ (tactic|
    first | positivity | exact norm_add_le _ _ | exact norm_sub_le _ _
    | exact real_inner_le_norm _ _
    | exact neg_norm_mul_norm_le_real_inner _ _))
| \ (tactic| inner_bound [$bound]) => do
  evalTactic (← \ (tactic|
    first | positivity | exact norm_add_le _ _ | exact norm_sub_le _ _
    | exact real_inner_le_norm _ _
    | exact neg_norm_mul_norm_le_real_inner _ _
    | exact norm_sq_le_norm_sq_of_le _ _ $bound
    | nlinarith [$bound]))
| \ (tactic| inner_bound [$bounds,*]) => do
  evalTactic (← \ (tactic|
    first | positivity | exact norm_add_le _ _ | exact norm_sub_le _ _
    | exact real_inner_le_norm _ _
    | exact neg_norm_mul_norm_le_real_inner _ _
    | nlinarith [$bounds,*]))
```

first 按稳定、专门的规则优先级尝试；最后才使用 nlinarith 组合给定条件。

inner_norm: 测试代码 (一)

```
import LeanHilbert.InnerNorm

open LeanHilbert

variable {H : Type*} [NormedAddCommGroup H] [InnerProductSpace ℝ H]

example (x y : H) (a : ℝ) :
  ||a • x + (1 - a) • y|| ^ 2 + a * (1 - a) * ||x - y|| ^ 2 =
  a * ||x|| ^ 2 + (1 - a) * ||y|| ^ 2 := by
  inner_norm

example (x y : H) : ||x + y|| ^ 2 + ||x - y|| ^ 2 = 2 * ||x|| ^ 2 + 2 * ||y|| ^ 2 := by
  inner_norm

example (x y : H) (a b : ℝ) :
  ||a • x + b • y|| ^ 2 =
  a ^ 2 * ||x|| ^ 2 + 2 * a * b * inner ℝ x y + b ^ 2 * ||y|| ^ 2 := by
  inner_norm
```

inner_norm: 测试代码 (二)

```
example (x y z : H) (a b c : ℝ) :  
  ||a • x + b • y + c • z|| ^ 2 =  
    a ^ 2 * ||x|| ^ 2 + b ^ 2 * ||y|| ^ 2 + c ^ 2 * ||z|| ^ 2 +  
    2 * a * b * inner ℝ x y + 2 * a * c * inner ℝ x z + 2 * b * c * inner ℝ y z := by  
  inner_norm
```

```
example (x y z : H) (a b c : ℝ) :  
  ||a • x + b • y + c • z|| ^ 2 =  
    ||z|| ^ 2 * c * c + a * a * ||x|| ^ 2 + b * ||y|| ^ 2 * b +  
    inner ℝ y x * b * 2 * a + c * inner ℝ z x * a * 2 +  
    b * c * 2 * inner ℝ z y := by  
  inner_norm
```

```
example (x y : H) : inner ℝ x y = (||x + y|| ^ 2 - ||x - y|| ^ 2) / 4 := by  
  inner_norm
```

```
example (x y z : H) (a b c : ℝ) :  
  ||a • (x - y) + b • (y - z) + c • (z - x)|| ^ 2 =  
    (a - c) ^ 2 * ||x|| ^ 2 + (b - a) ^ 2 * ||y|| ^ 2 + (c - b) ^ 2 * ||z|| ^ 2 +  
    2 * (a - c) * (b - a) * inner ℝ x y +  
    2 * (a - c) * (c - b) * inner ℝ x z +  
    2 * (b - a) * (c - b) * inner ℝ y z := by  
  inner_norm
```

inner_bound: 测试代码 (一)

```
import LeanHilbert.InnerBound

open LeanHilbert

variable {H : Type*} [NormedAddCommGroup H] [InnerProductSpace ℝ H]

example (x : H) : 0 ≤ ||x|| := by
  inner_bound

example (x y : H) : ||x + y|| ≤ ||x|| + ||y|| := by
  inner_bound

example (x y : H) : -||x|| * ||y|| ≤ inner ℝ x y := by
  inner_bound

example (x y : H) (h : ||x|| ≤ ||y||) : ||x|| ^ 2 ≤ ||y|| ^ 2 := by
  inner_bound [h]
```

inner_bound: 测试代码 (二)

```
example (x y : H) : inner ℝ x y + inner ℝ y x ≤ 2 * ||x|| * ||y|| := by
  inner_bound [real_inner_le_norm x y, real_inner_le_norm y x]
```

```
example (x y z : H) : ||x + y + z|| ≤ ||x|| + ||y|| + ||z|| := by
  inner_bound [norm_add_le (x + y) z, norm_add_le x y]
```

```
example (x : H) (hx : x ≠ 0) : 0 < ||x|| := by
  inner_bound [norm_pos_iff.mpr hx]
```


- ① 问题与动机
- ② Tactic 设计
- ③ 论文应用与实现

论文应用：原始显式证明（一）

```

theorem km_descent_inequality_old
  (T : H → H) (x y : H) (a : ℝ)
  (hT : ∀ u v : H, ||T u - T v|| ≤ ||u - v||)
  (hy : T y = y) (ha : 0 ≤ a) :
  ||(1 - a) • x + a • T x - y|| ^ 2 ≤ ||x - y|| ^ 2 - a * (1 - a) * ||T x - x|| ^ 2 := by
have h_contraction : ||T x - y|| ≤ ||x - y|| := by
  simp only [hy] using hT x y
have h_sq : ||T x - y|| ^ 2 ≤ ||x - y|| ^ 2 :=
  pow_le_pow_left0 (norm_nonneg _) h_contraction 2
have h_expand :
  ||(1 - a) • (x - y) + a • (T x - y)|| ^ 2 =
    (1 - a) * ||x - y|| ^ 2 + a * ||T x - y|| ^ 2 -
    a * (1 - a) * ||(T x - y) - (x - y)|| ^ 2 := by
rw [← real_inner_self_eq_norm_sq ((1 - a) • (x - y) +
  a • (T x - y)), ← real_inner_self_eq_norm_sq (x - y),
  ← real_inner_self_eq_norm_sq (T x - y),
  ← real_inner_self_eq_norm_sq ((T x - y) - (x - y))]
simp only [inner_add_left, inner_add_right, inner_sub_left,
  inner_sub_right, real_inner_smul_right, real_inner_comm]
ring

```

论文应用：原始显式证明（二）

```
have h_step : (1 - a) • x + a • T x - y =  
  (1 - a) • (x - y) + a • (T x - y) := by  
  module  
have h_difference : (T x - y) - (x - y) = T x - x := by  
  abel  
rw [h_step, h_expand, h_difference]  
have h_weighted : a * ||T x - y|| ^ 2 ≤ a * ||x - y|| ^ 2 :=  
  mul_le_mul_of_nonneg_left h_sq ha  
calc  
  (1 - a) * ||x - y|| ^ 2 + a * ||T x - y|| ^ 2 -  
    a * (1 - a) * ||T x - x|| ^ 2 =  
  a * ||T x - y|| ^ 2 + (1 - a) * ||x - y|| ^ 2 -  
    a * (1 - a) * ||T x - x|| ^ 2 := by ring  
_ ≤ a * ||x - y|| ^ 2 + (1 - a) * ||x - y|| ^ 2 -  
    a * (1 - a) * ||T x - x|| ^ 2 :=  
  sub_le_sub_right  
    (by simpa [add_comm] using  
      add_le_add_left h_weighted ((1 - a) * ||x - y|| ^ 2)) _  
_ = ||x - y|| ^ 2 - a * (1 - a) * ||T x - x|| ^ 2 := by ring
```

论文应用：使用两个 tactic 的证明

```
theorem km_descent_inequality
  (T : H → H) (x y : H) (a : ℝ)
  (hT : ∀ u v : H, ||T u - T v|| ≤ ||u - v||)
  (hy : T y = y) (ha : 0 ≤ a) :
  ||(1 - a) • x + a • T x - y|| ^ 2 ≤ ||x - y|| ^ 2 - a * (1 - a) * ||T x - x|| ^ 2 := by
have h_contraction : ||T x - y|| ≤ ||x - y|| := by
  simpa only [hy] using hT x y
have h_sq : ||T x - y|| ^ 2 ≤ ||x - y|| ^ 2 := by
  inner_bound [h_contraction]
have h_expand :
  ||(1 - a) • (x - y) + a • (T x - y)|| ^ 2 =
  (1 - a) * ||x - y|| ^ 2 + a * ||T x - y|| ^ 2 -
  a * (1 - a) * ||T x - x|| ^ 2 := by
  inner_norm
have h_step : (1 - a) • x + a • T x - y =
  (1 - a) • (x - y) + a • (T x - y) := by
  module
rw [h_step, h_expand]
inner_bound [mul_le_mul_of_nonneg_left h_sq ha]
```

代码长度与证明结构的对照

版本	证明主体	代码量	主要工作
原始显式证明	km_descent_inequality_old	约 50 行	范数平方转内积、双线性展开、手工 calc 重排与不等式传递
tactic 版本	km_descent_inequality	约 16 行	inner_norm 处理恒等式, inner_bound 结合已知界处理平方比较和最终不等式

结论

代码不仅更短，也将“内积范数代数”和“关键数学思路”分离：后续同类证明可以直接复用 tactic。

实现机制：将数学规则映射为 Lean 的自动化管线

inner_norm

- ① 用 `real_inner_self_eq_norm_sq` 消去范数平方；
- ② `simp only` 展开内积的加法、减法、负号与实数数乘；
- ③ 用 `real_inner_comm` 统一交叉项；
- ④ `ring` 完成多项式恒等式。

inner_bound

- ① 优先匹配 `positivity`、三角不等式和 Cauchy–Schwarz 界；
- ② 给定一个已知界时，也尝试“范数比较 $\Rightarrow$ 平方比较”；
- ③ 给定多个已知界时，交给 `nlinarith` 组合。

两者互补：前者规范化等式，后者闭合或组合不等式。

结论与可复用价值

- 以 KM 下降不等式为出发点，定位了形式化证明中的内积-范数代数瓶颈；
- `inner_norm` 将实希尔伯特空间中的平方范数恒等式化为标准代数计算；
- `inner_bound` 将常用范数/内积界与用户提供的外部证明统一组合；
- 测试中多个典型恒等式与不等式均已由 Lean 成功检验，论文核心证明也更短、更可读。

谢谢！